

Network-Based Laboratory Exam System: A Distributed LAN Architecture for Real-Time Monitoring, Secure Submission, and Operator-Controlled Exam Sessions

Şevval Naz Özkaraca
Dept. of Computer Engineering
İstanbul Kültür University
Istanbul, Türkiye
2300008192@stu.iku.edu.tr

Barış Başbekleyen
Dept. of Computer Engineering
İstanbul Kültür University
Istanbul, Türkiye
2300005352@stu.iku.edu.tr

Ahmet Deniz Sezgin
Dept. of Computer Engineering
İstanbul Kültür University
Istanbul, Türkiye
2300005633@stu.iku.edu.tr

Engin Erkurt
Dept. of Computer Engineering
İstanbul Kültür University
Istanbul, Türkiye
2300006437@stu.iku.edu.tr

Rana Neslihan Kurt
Dept. of Computer Engineering
İstanbul Kültür University
Istanbul, Türkiye
2300006496@stu.iku.edu.tr

İrem Su Ayrancınar
Dept. of Computer Engineering
İstanbul Kültür University
Istanbul, Türkiye
2300005623@stu.iku.edu.tr

Mert Özek
Dept. of Computer Engineering
İstanbul Kültür University
Istanbul, Türkiye
2300007579@stu.iku.edu.tr

Abstract—Computer-based laboratory examinations require infrastructure that is reliable on commodity classroom hardware, observable enough for academic integrity enforcement, and resilient against transient network failures. This paper presents the Network-Based Laboratory Exam System (NBLES), a self-contained client-server platform designed to supervise paper-replacement examinations on a single classroom local area network. The system runs as a single authoritative server process that coordinates exam lifecycle, persistent user state, policy distribution, and submission collection, while one client process per student handles authentication, local monitoring, incident reporting, and final upload. The runtime protocol uses checksum-protected JSON over HTTP and WebSocket, with HMAC signing and optional Fernet encryption for sensitive events. Monitoring covers process activity, focused-window changes, hardware deltas, idle behaviour, and on-demand screen replay. A normalized incident-rule engine classifies observations as whitelist, warning, or blacklist and applies operator-configured actions including pause, kick, ban, and remote kill-pid. Persistence is implemented as a five-table SQLite store and a five-layer JSONL audit trail that supports post-exam timeline reconstruction. The implementation totals approximately thirty thousand lines of Python and passes one hundred forty-nine of one hundred fifty-one automated tests; the two outstanding failures are isolated to a Windows-path mock fixture and do not affect production code paths.

Index Terms—exam monitoring, local area network, WebSocket, distributed systems, incident detection, secure submission, audit trail, software engineering project

I. INTRODUCTION

Computer-based laboratory examinations require infrastructure that is simultaneously reliable on commodity classroom hardware, observable enough for academic integrity enforcement, and resilient against transient network failures common in university lab environments. Commercial proctoring solutions are typically cloud-based, require continuous internet connectivity, and expose student data to third-party providers. Such trade-offs are often unacceptable in a controlled lab where the institution wants to retain full data custody and run the entire stack on a local area network. (*Joint authorship*)

This work describes a self-contained exam runtime that runs entirely on a single classroom LAN. One authoritative server process binds to a configurable host and port and announces itself via a UDP discovery beacon. Student clients locate the server either by an explicit host and port or by an automatic discovery exchange, authenticate against an allowed-user list and an optional HMAC-signed token, and open a WebSocket runtime channel. During the exam, the client owns local observation because only the student machine can produce that data; the server owns authoritative session state, policy distribution, and the submission archive. (*Joint authorship*)

The contributions of this paper are: (i) a layered architecture that cleanly separates the LAN runtime protocol from same-machine loopback inter-process communication, preventing

dashboard-control semantics from leaking into the student protocol; (ii) a normalized incident-rule engine shared by client and server with whitelist, warning, and blacklist classifications and operator-configurable actions; (iii) an authentication subsystem combining CATS preflight, Active Directory token validation, and a temporary-disable workflow gated by explicit admin approval, with passwords never persisted in the validation queue; (iv) a privacy-safe read-only projector view for in-room notification display; (v) a five-table SQLite persistence layer with a five-layer JSONL audit trail and a chronological reconstruction query for post-exam review; and (vi) a documented testing strategy producing 151 automated tests across unit, integration, and system layers. The project is developed under COM6064 Software Engineering at İstanbul Kültür University by a seven-member team with complementary roles, summarized in Table I. The remainder of this paper is organized as follows: Section II reviews related work; Section III describes the layered architecture; Section IV presents the requirements analysis; Section V details the design and implementation; Section VI discusses security and privacy; Section VII reports testing and validation; Section VIII presents results and discussion; Section IX concludes. *(Joint authorship)*

TABLE I
TEAM ROLES AND PRIMARY CONTRIBUTIONS

Member	Primary Role
Şevval Naz Özkaraca	Secure transmission, authentication preflight
Barış Başbekleyen	Server runtime, exam lifecycle, admin commands
Ahmet Deniz Sezgin	Client runtime, monitoring pipeline, submission
Engin Erkurt	Service discovery, network reliability, buffering
Rana Neslihan Kurt	Reusable Qt UI library, policy management UI
İrem Su Ayrancınar	Main dashboard, live violation supervision
Mert Özek	Persistence schema, audit-trail extension

II. RELATED WORK

Network-based examination supervision sits at the intersection of three established lines of work. Commercial cloud-proctoring services such as ProctorU, Respondus, and Honorlock provide identity verification and remote observation but require continuous internet connectivity and externalize student data to third-party providers [1]. Classroom-management toolkits such as Veyon [9], Epoptes [10], and the legacy Mythware E-Classroom Management Software target the in-room supervision use case but historically lack a formal incident-rule engine and a defensible post-exam audit trail. Activity-monitoring frameworks such as ActivityWatch [11] concentrate on per-user telemetry rather than classroom-wide enforcement. *(Joint authorship)*

The runtime layer of NBLES draws on standard distributed-systems techniques. WebSocket-based real-time channels [2], message integrity through SHA-256 and HMAC [4], and HTTP for stateless one-off operations [3] are well-understood building blocks; the contribution here is the disciplined separation between the LAN runtime plane and the same-host loopback IPC plane so that administrative semantics never

leak into the protocol exposed to student hosts. *(Şevval Naz Özkaraca, Engin Erkurt)*

Academic-integrity research increasingly emphasises post-exam reviewability in addition to live observation [12]. NBLES inherits this emphasis by adopting an append-only five-layer audit trail with explicit causal links between raw observations and interpreted violations, and by exposing the reconstruction query through RBAC-gated HTTP endpoints designed for the post-exam reviewer rather than for the live runtime. *(Mert Özek)*

III. SYSTEM ARCHITECTURE

NBLES follows a layered architecture organized into four top-level Python packages: a shared layer (*common/*), a server-side runtime (*server/*), a client-side runtime (*client/*), and a set of operating-system observers (*client/custommodules/*). The layout is designed so that each package has one owner and a stable contract with its neighbours; cross-package imports are unidirectional, and no cycle is permitted. *(Joint authorship)*

A. Process Topology

The deployment consists of one server host running the server entry-point process and any number of student hosts running the client entry-point process. Each host may additionally spawn local UI processes: the server spawns a Tk or Qt operator dashboard, and the client spawns a Tk or Qt timer and submission window. Launcher processes supervise the CLI processes, restart them on graceful exit, and surface their stderr in a side panel. The server additionally serves a read-only HTML projector view that the operator opens in any browser and duplicates onto the classroom projector. *(Barış Başbekleyen, İrem Su Ayrancınar)*

B. Two Communication Planes

The system intentionally exposes two distinct communication planes. The LAN runtime plane uses JSON-over-HTTP for one-off operations such as login, exam-config fetch, file download, submission upload, artifact upload, health check, and authentication-status query, and JSON-over-WebSocket for the continuous exam runtime. This plane is bound to the configured server host and port and is the only channel between server and student. The local IPC plane uses a loopback-only WebSocket on 127.0.0.1 with an ephemeral port and a per-process random token, used between same-machine CLI, GUI, and launcher processes. A stdin fallback is preserved for manual terminal operation. This separation is enforced by binding the local IPC server only to 127.0.0.1, validating loopback peers explicitly, and refusing tokens that do not match the per-process secret passed through environment variables. *(Şevval Naz Özkaraca, Barış Başbekleyen)*

C. Package Responsibility Model

The *common/* package contains the protocol envelope and checksum, event constructors, the session security context, UDP discovery, the loopback IPC server and client, process and incident-rule definitions, text-safety helpers, and runtime

logging. The *server/* package implements HTTP and WebSocket handlers, persistent state, the admin command parser and dispatcher, the time-broadcast loop, the GUI bridge, the settings service, the shutdown routine, the projector endpoint, and the persistence and audit-trail extensions. The *client/* package handles the discovery and login flow, exam preparation, the WebSocket session, monitor orchestration, the incident engine, the durable incident buffer, the transfer and upload pipeline, and the student-facing GUI bridge. The *client/custommodules/* package provides the process monitor, the focused-window monitor, the hardware monitor, the idle monitor, and the replay recorder. (Joint authorship)

[Figure 1: Server bootstrap output showing duplicate-server precheck, route registration, discovery announcer startup, and the configured listening address.]

IV. REQUIREMENTS ANALYSIS

A. Functional Requirements

A complete Software Requirements Specification with stable requirement identifiers is provided as a companion document. The requirements are grouped by area. FR-SRV-* covers server runtime, including CLI argument validation, the duplicate-server guard, route registration, persistent state, the session state machine, the time broadcast, the admin command dispatcher, banned-user safety during global finish, and WebSocket close-frame size compliance. FR-CLI-* covers client runtime: discovery or direct connection, login, the preflight check, exam preparation, timer UI synchronisation, and monitor lifecycle across reconnect. (Barış Başbekleyen, Ahmet Deniz Sezgin)

FR-MON-* covers monitoring: process snapshots with blacklist matching, focused-window detection with Unicode-normalised window titles, idle detection with configurable thresholds, hardware deltas, and on-demand screen replay through FFmpeg. FR-INC-* and FR-POL-* govern the incident-rule engine and the policy distribution mechanism. FR-REC-* covers reconnect and durable buffering. FR-SUB-* covers submission and artifact transfer with per-file and archive SHA-256 verification. FR-AUTH-* covers CATS and Active Directory preflight, temporary-disable workflows, and HMAC token verification. FR-IPC-* and FR-UI-* cover local IPC and the Tk/Qt dashboard parity contract. FR-PROJ-* defines the projector page and the Server-Sent-Events feed with public-safe payloads. FR-DB-* and FR-AUDIT-* cover the persistence schema and the five-layer audit trail. (Ahmet Deniz Sezgin, Mert Özek)

B. Non-Functional Requirements

Non-functional requirements cover security, privacy, reliability, observability, usability, and operability. The security requirements mandate loopback-only local IPC, HMAC-signed and optionally Fernet-encrypted sensitive LAN events, safe ZIP extraction, and configurable upload size limits. Privacy requirements forbid personally identifying and evidentiary data on the projector payload. Reliability requirements treat reconnect as a network condition rather than a logging shutdown

and require atomic temp-file-then-rename writes for every editable persistence file. Observability requirements demand per-monitor JSONL logs plus per-process standard-output and standard-error capture. Usability requirements demand parity between the Tk and Qt dashboard back-ends. Operability requirements demand that the system be rebuildable from a single requirements.txt with no external services. (Şevval Naz Özkara, İrem Su Ayranpınar)

V. DESIGN AND IMPLEMENTATION

A. LAN Runtime Protocol

All WebSocket messages share the three-field envelope {event, data, checksum}, where the checksum is SHA-256 computed over the canonical JSON serialisation of the event name and the data object. Malformed envelopes are rejected before any handler logic runs. Sensitive events — exam policy distribution, policy update, session state, incident report, kill process, pause exam, and resume exam — are additionally wrapped by a session security context that signs the envelope with HMAC over canonical fields, attaches a timestamp and a nonce, optionally encrypts the payload with Fernet, and enforces a server-side replay window. (Şevval Naz Özkara)

B. Server Subsystems

The server uses aiohttp for HTTP, WebSocket, and Server-Sent Events. The application factory registers ten routes covering health, authentication status, login, exam configuration, exam files, exam submission, client artifact upload, the projector page, the projector event stream, and the WebSocket runtime endpoint. Background tasks include the time broadcast loop, the console reader, the discovery announcer, and the duplicate-server guard. Persistent user state lives in a JSON document keyed on login identifier; policy, blacklist, process definitions, and incident-rule files are companion JSON documents. (Barış Başbekleyen)

The server-side session state machine resolves a live status across eight named states from raw user fields and provides a single mutation entry point that keeps legacy boolean flags consistent. Admin commands are parsed by a single dispatcher and shared by stdin input and dashboard IPC so the CLI and dashboard cannot diverge. The command catalogue covers thirty-eight commands spanning timer and exam control, user moderation, settings management, monitoring requests, and the discovery announcer and duplicate-server-guard wiring. (Barış Başbekleyen)

C. Client Subsystems

The client orchestration entry point owns the WebSocket connection, the timer GUI bridge, the standard-input bridge, the incident engine, the incident buffer, the replay save queue, and four monitors — process, focused-window, hardware, and idle. Monitors are not stopped on transient WebSocket disconnects; they continue to log to local JSONL files, and unsent incidents accumulate in a durable buffer that flushes in order after reconnect. (Ahmet Deniz Sezgin, Engin Erkurt)

[Figure 2: Operator dashboard during an active exam session showing connected students, session state, incident counters, and the command console.]

D. Incident Rule Engine

Incident rules are normalised JSON definitions matched against incoming incidents using process-name, browser-process-name, and window-title patterns, with both *contains* and *exact* matching modes available. A best-match resolver chooses among matching rules by priority. A whitelist match suppresses the incident before any warning or blacklist evaluation is attempted. Warning and blacklist matches attach the matched rule, the matched rule identifier, and the configured-actions list to the outgoing report, allowing the server to apply ban, kick, pause, or kill-pid actions where the context permits. (Ahmet Deniz Sezgin, Rana Neslihan Kurt)

E. Persistence Schema and Audit Trail

The persistence layer is a thread-safe SQLite store with five tables: exam sessions, student connections, violations, monitoring events, and an audit log. The store is guarded by a reentrant lock so that the asynchronous timer task and the synchronous WebSocket handlers can call into it concurrently without corruption. On top of this layer, a five-layer JSONL audit store rooted at a dedicated audit directory provides an immutable session-header layer, an append-only raw-event stream, an interpreted-violation layer with explicit causal pointers back to the raw events that triggered each violation, a per-day instructor-action log, and a replace-in-place review-decision layer with full revision history. A merging query produces a chronologically ordered per-student timeline keyed on the observation timestamp, exposed through three RBAC-gated HTTP endpoints for the post-exam reviewer. Editable layers use a temp-file-then-rename atomic write so a crash mid-write cannot leave a half-written file, and every layer is written in UTF-8 so Turkish characters in user names, session reasons, and review notes are preserved verbatim. (Mert Özek)

F. Reusable UI Components

The instructor dashboard is built on top of a reusable Qt component library that supplies alert cards with severity variants, status badges for connection, security, and violation states, timeline rows, and summary table widgets. The same components are reused in the post-exam review screen so the operator and the reviewer see consistent visual vocabulary. The PySide6 implementation includes a custom QSS theme with glassmorphism panels and a dark accent palette tuned for high-density data displays. (Rana Neslihan Kurt, İrem Su Ayranpınar)

VI. SECURITY AND PRIVACY

A. Threat Model and Controls

The threat model assumes the LAN is trusted between the server and the network switches but that student machines may be uncooperative or actively adversarial. Controls layered against this model include allowed-user gating, HMAC token

validation, optional CATS and Active Directory preflight, a temporary-disable workflow that requires explicit operator approval and stores no passwords, a SHA-256 checksum on every LAN message, signed and optionally encrypted sensitive events with timestamp, nonce, and replay window, loopback-only token-protected local IPC, safe ZIP extraction with manifest-tracked managed folders, configurable upload size limits and SHA-256 verification on every upload, and a public-safe projector payload that excludes login identifiers, UUIDs, IP addresses, process names, window titles, paths, and evidence references. (Şevval Naz Özkaraca)

B. Authentication Subsystem

The client-side authentication flow checks whether CATS and Active Directory validation are required before student credentials are submitted to the server. The preflight query determines the current authentication configuration; if the status endpoint cannot be reached, the client falls back to strict local behaviour rather than silently bypassing the check. The admin-managed credential-validation queue stores the login identifier, the source IP, timestamps, and the disabled-auth modes but never the submitted password. A login-block gate covering banned, kicked, submitted, and finished states ensures completed or forbidden users cannot re-enter after lifecycle exit. (Barış Başbekleyen, Şevval Naz Özkaraca)

C. Security Logging

Security log entries capture events of interest for post-exam audit, including session establishment and teardown, token verification outcomes, temporary authentication-disable activations and expirations, admin validation queue approvals and denials, and unauthorised access attempts such as rejected loopback-token checks and blocked re-login attempts by lifecycle-exited users. The logs are written to the shared runtime-logging infrastructure alongside the per-monitor JSONL files. (Şevval Naz Özkaraca, Mert Özek)

VII. TESTING AND VALIDATION

Validation combines an automated suite with a manual smoke checklist. The automated suite contains twenty-nine unit tests, four integration tests, and two system tests for a total of one hundred fifty-one tests across the three layers. The most recent execution reported one hundred forty-nine passing tests; the two failures are isolated to a Windows-path mock fixture in the replay-recorder unit tests and do not affect production code paths. (Joint authorship)

Three production bugs were discovered, fixed, and regression-tested immediately before the integration milestone. First, a global finish was inadvertently clearing the banned flag on banned users via the legacy-flag synchronisation helper; the fix guards the loop with an explicit banned-state check, covered by the new regression test *test_finish_exam_global_skips_banned_user*. Second, some kick and forced-disconnect paths constructed WebSocket close reasons exceeding the RFC 6455 one-hundred-twenty-five-byte control-frame limit, and naïve byte slicing risked

splitting multi-byte UTF-8 code points; a safe close-message helper now walks UTF-8 boundaries safely and is covered by three new tests. Third, error responses for malformed events were sent through a raw send call while an active session security context existed, causing the client to fail decoding; the fix wraps the error envelope with the protect-payload helper so it respects the active security boundary. Cumulatively these fixes raised the automated test pass rate from one hundred sixteen of one hundred eighteen to one hundred forty-nine of one hundred fifty-one. (*Barış Başbekleyen*)

Critical regression areas explicitly covered by the test suite are summarised in Table II.

TABLE II
CRITICAL REGRESSION AREAS

Critical Regression Area
Global finish must not clear the banned flag on banned users.
WebSocket close reasons must respect the 125-byte control-frame limit.
Secured error responses on malformed events must remain inside the session security context.
Reconnect must not stop local monitoring or replay.
The buffered-incident restore path must deduplicate by sequence identity.
Exam ZIP extraction must not delete unmarked user folders.
Authentication disable must expire on schedule and require admin validation before login proceeds.
Projector payload must remain free of personally identifying or evidentiary data.

VIII. RESULTS AND DISCUSSION

[Figure 3: Student timer window during a running exam. Remaining time and session state are synchronised with the server through the LAN WebSocket runtime.]

The integration milestone exercised the full end-to-end flow from server startup with the duplicate-guard, through multi-client discovery and login, exam start, mid-exam process incident with replay request and artifact upload, transient WiFi disconnect with monitor continuity and buffered incident flush, global finish with banned-user safety, and final submission upload with SHA-256 verification. The complete walk-through ran to completion without operator intervention and produced a post-exam audit trail that the reviewer could replay through the reconstruction query. (*Joint authorship*)

The monitoring layer proved reliable under realistic laboratory conditions. Local process monitoring, focused-window monitoring, idle detection, hardware logging, JSONL persistence, timer GUI communication, replay recording, and incident processing all continued during WebSocket reconnection. Durable queued incident records with stable sequence metadata, ordered reconnect flush, acknowledgement-based removal, and deduplicated restore reduced incident loss and preserved violation evidence across reconnects. (*Engin Erkurt, Ahmet Deniz Sezgin*)

[Figure 4: Incident propagation from the client to the operator dashboard. A focused-window or process-blacklist match is normalised by the incident-rule engine and surfaced as an active row in the dashboard incident panel.]

The operator dashboard remained usable under high-frequency exam updates through row-identity and row-fingerprint refresh behaviour, in-place cell updates when row order is stable, scroll and selection restoration when rebuilds are required, and full-row hover feedback for both the Tk and Qt back-ends. Windows 11 display-scaling issues encountered earlier in integration were resolved during the final integration phase and validated through rigorous smoke testing of the unified interface. (*İrem Su Ayrancılar, Rana Neslihan Kurt*)

The persistence layer and audit-trail extension produced the expected five-layer artefact tree on disk: per-session immutable headers, the append-only raw-event stream, the violation layer with causal pointers back to the raw events, the per-day instructor-action log, and the replace-in-place review-decision layer with full revision history. The reconstruction query returned a chronologically ordered timeline for every probed session, with the observation timestamp preferred over the recording timestamp so out-of-order writes are still ordered correctly. (*Mert Özek*)

[Figure 5: Read-only projector view. The payload exposes only aggregate counts, exam phase, and generic notifications, intentionally omitting login identifiers, UUIDs, IP addresses, and evidence paths.]

[Figure 6: Final submission flow on the student client. Upload success is shown only after the server confirms the stored archive checksum.]

IX. CONCLUSION AND FUTURE WORK

This paper presented a complete LAN-resident exam run-time system covering authentication, exam lifecycle, multi-dimensional monitoring, incident classification, durable buffering, secure submission, a five-table SQLite persistence layer, a five-layer JSONL audit trail with chronological reconstruction, and a public-safe projector view. The architecture deliberately separates the student-facing LAN protocol from same-machine loopback IPC, which keeps administrative semantics out of the protocol exposed to student hosts. The implementation has been validated through one hundred fifty-one automated tests and a documented manual smoke checklist; the remaining mock-related test failures will be addressed in the final delivery. (*Joint authorship*)

Future extensions include stronger authentication integration with centralised identity providers, signed settings bundles with policy provenance tracking, more precise browser observation through controlled extensions or accessibility APIs, an automated Windows packaging pipeline, an operator audit-log export format, per-course and per-exam profile management, and optional encrypted-at-rest storage for submissions and artefacts. A read-only replica of the persistence store for stricter consistency under high concurrent load is also a natural next step. (*Joint authorship*)

ACKNOWLEDGMENT

The authors thank Prof. Dr. Akhan Akbulut and Teaching Assistant Arş. Gör. Büşra Koçaçınar Öz for their guidance throughout COM6064 Software Engineering at İstanbul Kültür University.

REFERENCES

- [1] A. Tanenbaum and D. Wetherall, *Computer Networks*, 5th ed., Boston, MA: Pearson, 2011.
- [2] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, Dec. 2011.
- [3] R. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing," RFC 7230, IETF, Jun. 2014.
- [4] D. Eastlake and T. Hansen, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)," RFC 6234, IETF, May 2011.
- [5] Python Software Foundation, "aiohttp — Asynchronous HTTP client/server framework," [Online]. Available: <https://docs.aiohttp.org/>.
- [6] G. Rodola, "psutil documentation," [Online]. Available: <https://psutil.readthedocs.io/>.
- [7] Python Cryptographic Authority, "cryptography library documentation," [Online]. Available: <https://cryptography.io/>.
- [8] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.
- [9] Veyon Project, "Veyon: cross-platform classroom-management and computer-monitoring framework," [Online]. Available: <https://github.com/veyon/veyon/>.
- [10] Epoptes Project, "Epoptes: open-source classroom management and supervision for Linux," [Online]. Available: <https://github.com/epoptes/epoptes>.
- [11] ActivityWatch Project, "ActivityWatch: cross-platform automated time-tracking and activity logging," [Online]. Available: <https://github.com/ActivityWatch/activitywatch>.
- [12] International Center for Academic Integrity, "The Fundamental Values of Academic Integrity," 3rd ed., 2021. [Online]. Available: <https://academicintegrity.org/>.
- [13] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Upper Saddle River, NJ: Pearson, 2016.
- [14] ISO/IEC/IEEE, "Systems and software engineering — Life cycle processes — Requirements engineering," ISO/IEC/IEEE 29148:2018, 2018.
- [15] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Reading, MA: Addison-Wesley, 1994.
- [16] R. T. Fielding, "Architectural Styles and the Design of Network-Based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.
- [17] FFmpeg Project, "FFmpeg documentation," [Online]. Available: <https://ffmpeg.org/documentation.html>.
- [18] SQLite Consortium, "SQLite documentation," [Online]. Available: <https://www.sqlite.org/docs.html>.
- [19] The Qt Company, "PySide6: Official Python bindings for Qt 6," [Online]. Available: <https://doc.qt.io/qtforpython-6/>.
- [20] EmpCloud, "emp-monitor: open-source employee activity monitoring tool," [Online]. Available: <https://github.com/EmpCloud/emp-monitor>.
- [21] Gorav22, "AI-Proctor-Exam-Monitor: AI-assisted online exam proctoring monitor," [Online]. Available: <https://github.com/Gorav22/AI-Proctor-Exam-Monitor>.
- [22] henrypp, "simplewall: lightweight Windows firewall with per-application rules," [Online]. Available: <https://github.com/henrypp/simplewall>.
- [23] "Online Exam Monitoring System," *International Journal of Engineering Development and Research*, vol. 8, no. 3, 2020. [Online]. Available: <https://rjwave.org/ijedr/papers/IJEDR2003018.pdf>.